AuraQ

AI Customer Quality Auditor

End-to-End Project Document

Document version v1.0.0	Last Updated 26 March,2026	Author: Gunda Lakshmi Gayathri

Executive Summary

AuraQ is an AI-powered Customer Quality Auditor that automates the evaluation of customer-agent interactions through audio and text input. It uses Deepgram for transcription, Groq LLaMA 3.1 8B for quality scoring across empathy, compliance, and resolution dimensions, and a LangChain + Pinecone RAG pipeline for contextual audit intelligence. The platform features customer emotion detection, satisfaction scoring, bias analysis, and a React dashboard with charts, call history, and downloadable reports. Built by a three-member Agile team across 4 sprints, the system is fully deployed on Vercel (frontend) and Render (backend). AuraQ enables organisations to review 100% of customer interactions with consistency, speed, and fairness at scale.

Attribute	Details	Value
Project name	AuraQ - AI Customer Quality Auditor	v1.0.0
Developer	Gunda Lakshmi Gayathri	Infosys Intern 2026
Timeline	Feb 2 - Mar 26,2026	4 sprints, 3 Dev
Status	Production Ready	Deployed on Render+Vercel
License	MIT License	©Gunda Lakshmi Gayathri

1. Project Overview

1.1 Problem Statement

Customer support organisations rely heavily on manual quality assurance processes to evaluate agent performance, where supervisors randomly sample only 5–10% of interactions for review. This approach is time-consuming, inconsistent, and highly subjective — leading to biased agent evaluations, missed compliance violations, and a lack of real-time visibility into customer sentiment. Without automated intelligence, organisations struggle to maintain consistent service quality, identify coaching opportunities, and make data-driven decisions at scale..

1.3 Project Objectives

- To automate the end-to-end QA review process by enabling audio call and text/chat log uploads with AI-powered transcription and summarisation.
- To provide standardised, objective quality scoring across three dimensions — empathy, compliance, and resolution — using large language models on a 1–10 scale.
- To detect customer emotions across 8 categories and compute a satisfaction percentage to give supervisors real-time sentiment visibility.
- To ensure fair and unbiased agent evaluations through a dedicated bias and fairness analysis module with 4 sub-scores.
- To enable policy-aware scoring and contextual transcript retrieval using a RAG pipeline built on LangChain and Pinecone.
- To deliver a user-friendly React dashboard with visual analytics, call history, batch upload, and downloadable audit reports.
- To deploy the platform on the cloud for 24/7 accessibility with uptime monitoring.

1.4 Scope

In Scope	Out of Scope
• Audio call upload and transcription • Text/chat log upload and summarisation • LLM-based quality scoring (3 dimensions) • Emotion detection and satisfaction scoring • Bias/fairness analysis • RAG-powered contextual summaries • Policy document ingestion via RAG pipeline • React dashboard with charts and history • PDF/DOC report generation • Batch audio upload • Cloud deployment (Render + Vercel)	• User authentication / role-based login • Real-time live call monitoring • Multi-tenant / multi-organisation support • Native mobile application • CRM / helpdesk integrations (Salesforce, Zendesk) • Advanced analytics with predictive modelling

2. Team & Roles

AuraQ was built by a team of three developers, each owning distinct technical domains and collaborating across sprints.

Team Member	Role	Primary Responsibilities
Gunda Lakshmi Gayathri	Full-Stack Developer	Text/chat server (chat_app.py), emotion detection, satisfaction scoring, React dashboard, mobile responsiveness, PDF/DOC reports, IST timestamps, Vercel deployment, server consolidation
Siddhi Dhamal	AI/ML Engineer & Backend Dev	Audio server, Deepgram integration, speaker diarisation, LLM scoring (empathy/compliance/resolution), RAG pipeline (LangChain + Pinecone), batch upload, Render deployment
Sowmya	QA & Integration Developer	Transcript/summary file storage, fairness analysis, Recharts visualisations, call history, UptimeRobot monitoring, GitHub MIT licence

3. Technology Stack

3.1 Architecture Overview

AuraQ follows a decoupled, microservices-inspired architecture. The frontend (React SPA) communicates with a unified FastAPI backend, which orchestrates calls to multiple AI services and manages local file storage. The backend consolidates audio transcription, text analysis, LLM scoring, and RAG-based retrieval under a single main.py entry point.

3.2 Technology Components

Layer	Technology	Purpose
Frontend	React + Vite	Single-page application (SPA) for the user-facing dashboard
Frontend	Tailwind CSS	Utility-first styling and responsive design
Frontend	Recharts	Interactive data visualisation charts for quality metrics
Backend	Python + FastAPI	RESTful API server — unified main.py with route merging
Transcription	Deepgram Nova-2	Audio call transcription with speaker diarisation (agent/customer split)
Summarisation	Deepgram Text Intelligence / summarise v2	Auto-generated summaries for audio transcripts and chat logs
LLM Scoring	Groq LLaMA 3.1 8B	Empathy, compliance, and resolution scoring on 1–10 scale
Emotion Detection	Groq LLaMA 3.1 8B	8-category customer emotion detection and 0–100% satisfaction scoring
RAG Pipeline	LangChain + Pinecone	Contextual retrieval from past transcripts; policy document ingestion
File Storage	Local disk (CSV-based)	Per-file transcript, summary, and score storage; call history
Reporting	Python PDF/DOC libs	Downloadable PDF and DOC audit reports per call file
Frontend Deploy	Vercel	Cloud hosting for the React SPA
Backend Deploy	Render	Cloud hosting for the FastAPI backend
Monitoring	UptimeRobot	5-minute ping to prevent cold-starts and ensure 24/7 uptime
Timezone	UTC+5:30 (IST)	Accurate timestamps for Indian users across all records

4. Features & Functional Description

4.1 Audio Call Upload & Transcription

Users can upload audio call recordings. AuraQ sends the audio to Deepgram Nova-2, which transcribes the call and performs speaker diarisation — automatically splitting the transcript into Agent and Customer turns. The structured transcript is saved per file for downstream scoring and retrieval.

4.2 Text / Chat Log Upload & Summarisation

Users can upload text-based chat logs in any speaker format. The dynamic chat parser (chat_app.py) normalises the input regardless of format. Deepgram Text Intelligence (summarise v2) generates an auto-summary of the interaction. Large texts are chunked and summarised in segments, then merged into a cohesive final summary.

4.3 LLM Quality Scoring Engine

The Groq LLaMA 3.1 8B model analyses each transcript and produces three standardised quality scores on a 1–10 scale:

- Empathy Score — measures the agent's emotional intelligence, understanding, and tone warmth. Accompanied by an empathy timeline across the call.
- Compliance Score — evaluates adherence to company protocols, scripts, and regulatory requirements. Includes a step-by-step compliance checklist breakdown.
- Resolution Score — assesses how effectively the agent identified and resolved the customer's issue.

4.4 Customer Emotion Detection & Satisfaction Scoring

AuraQ detects the customer's emotional state across 8 categories (e.g., Frustrated, Satisfied, Confused, Angry, Neutral, Happy, Anxious, Disappointed). A satisfaction percentage (0–100%) with a descriptive label (e.g., Highly Satisfied, At Risk) is computed to give supervisors an instant snapshot of the customer experience.

4.5 Bias & Fairness Analysis

To ensure equitable agent evaluations, AuraQ includes a fairness analysis module that produces 4 sub-scores assessing potential bias in the interaction and the evaluation. This feature helps QA teams identify systematic evaluation inconsistencies and supports a fair performance management culture.

4.6 RAG-Powered Contextual Intelligence

Using LangChain and Pinecone vector database, AuraQ enables:

- Contextual Retrieval — users can ask natural language questions about past call transcripts and receive precise, evidence-grounded answers.
- Policy Document Ingestion — supervisors can upload company policy documents. AuraQ runs them through the RAG pipeline so agent scores are calibrated against actual company policies, not generic rubrics.
- RAG Audit Summaries — instant AI-generated audit summaries that contextualise findings against historical call data and policy benchmarks.

4.7 React Dashboard

The React + Vite + Tailwind CSS dashboard provides:

- A unified metrics view showing all quality scores for each uploaded call.
- Interactive Recharts visualisations (bar charts, line charts, radar charts) for quality trends.
- Call History panel with IST timestamps and per-file summaries.
- Batch audio upload — process multiple files asynchronously in a single session.
- Mobile-responsive layout with bottom navigation for on-the-go access.

4.8 Report Generation

For each analysed call, AuraQ generates a downloadable report containing the full transcript, summary, all quality scores, emotion analysis, and fairness assessment. Reports are available in both PDF and DOC formats, ready to be shared with stakeholders or stored for compliance records.

5. Agile Project Management

AuraQ was developed using the Agile Scrum methodology over 4 sprints spanning 8 weeks (February–March 2026). Each sprint delivered a functional milestone, with daily stand-ups, sprint retrospectives, and impediment tracking throughout.

5.1 Sprint Summary

Sprint	Milestone	Start Date	End Date	Status
Sprint 1	Setup & Transcription – Audio + Text servers, Deepgram integration	02 Feb 2026	13 Feb 2026	Completed
Sprint 2	LLM Scoring Engine – Empathy, Compliance, Resolution, Emotion, Fairness	16 Feb 2026	27 Feb 2026	Completed
Sprint 3	RAG Pipeline & Dashboards – React UI, Charts, Batch Upload, LangChain+Pinecone	02 Mar 2026	13 Mar 2026	Completed
Sprint 4	Deployment & Monitoring – Server merge, Render + Vercel, UptimeRobot	16 Mar 2026	24 Mar 2026	Completed

** UptimeRobot monitoring (US-022) remains In Progress.*

5.2 Product Backlog (All User Stories)

US ID	User Story	Priority	Assignee	Status
US-001	As a user, I want to upload audio call files so they can be transcribed automatically	MUST HAVE	Siddhi Dhamal	Completed
US-002	As a user, I want speaker-separated transcripts so I can review conversations clearly	MUST HAVE	Siddhi Dhamal	Completed
US-003	As a user, I want to upload text/chat files so they can be summarised	MUST HAVE	G. L. Gayathri	Completed
US-004	As a user, I want auto-generated summaries so I save review time	MUST HAVE	G. L. Gayathri	Completed
US-005	As a supervisor, I want empathy scores so I can evaluate agent warmth	MUST HAVE	Siddhi Dhamal	Completed
US-006	As a supervisor, I want compliance scores so I can check protocol adherence	MUST HAVE	Siddhi Dhamal	Completed
US-007	As a supervisor, I want resolution scores so I can track issue resolution	MUST HAVE	Siddhi Dhamal	Completed
US-008	As a supervisor, I want customer emotion detection so I understand sentiment	MUST HAVE	G. L. Gayathri	Completed

US-009	As a supervisor, I want satisfaction percentage so I can measure happiness	MUST HAVE	G. L. Gayathri	Completed
US-010	As a user, I want fairness analysis so agent evaluation is unbiased	SHOULD HAVE	Sowmya	Completed
US-011	As a user, I want a React dashboard so I can view all metrics in one place	MUST HAVE	G. L. Gayathri / Siddhi	Completed
US-012	As a user, I want charts so I can visualise quality trends over time	SHOULD HAVE	Sowmya	Completed
US-013	As a user, I want call history so I can review past uploaded files	SHOULD HAVE	Sowmya	Completed
US-014	As a user, I want PDF/DOC reports so I can share results with my team	SHOULD HAVE	G. L. Gayathri	Completed
US-015	As a user, I want batch upload so I can process multiple audio files at once	SHOULD HAVE	Siddhi Dhamal	Completed
US-016	As a user, I want the app to work on mobile so I can access it anywhere	COULD HAVE	G. L. Gayathri	Completed
US-017	As a user, I want contextual retrieval on transcripts so I can ask questions about past calls	SHOULD HAVE	Sowmya	Completed
US-018	As a user, I want to upload policy documents and generate RAG-based scores	SHOULD HAVE	Siddhi Dhamal	Completed
US-019	As a supervisor, I want RAG-powered audit summaries for contextual insights	SHOULD HAVE	Siddhi Dhamal	Completed
US-020	As admin, I want all servers combined so deployment is simpler	MUST HAVE	G. L. Gayathri	Completed
US-021	As admin, I want cloud deployment so users can access the app anytime	MUST HAVE	G. L. Gayathri	Completed
US-022	As admin, I want uptime monitoring so the server stays awake 24/7	MUST HAVE	Siddhi Dhamal	In Progress
US-023	As admin, I want IST timestamps so file times are accurate for Indian users	SHOULD HAVE	G. L. Gayathri	Completed

6. Sprint Task Details

6.1 Sprint 1 — Setup & Transcription (Feb 2 – Feb 13, 2026)

Sprint 1 focused on establishing the core infrastructure: both the audio and text FastAPI servers, Deepgram integrations, speaker diarisation, dynamic chat parsing, and per-file transcript/summary storage.

Task	Description	Team Member	Est. Hours	Status
T-01	Set up FastAPI audio server (app.py)	Siddhi Dhamal	8	Completed
T-02	Integrate Deepgram Nova-2 for audio transcription	Siddhi Dhamal	5	Completed
T-03	Implement speaker diarisation (Agent/Customer split)	Siddhi Dhamal	5	Completed
T-04	Set up FastAPI text server (chat_app.py)	G. L. Gayathri	5	Completed
T-05	Implement dynamic chat parser for any speaker format	G. L. Gayathri	5	Completed
T-06	Integrate Deepgram Text Intelligence for summaries	G. L. Gayathri	3	Completed
T-07	Save per-file transcripts and summaries to disk	Sowmya	3	Completed
T-08	Chunked summarisation for large text inputs	Sowmya	2	Completed

6.2 Sprint 2 — LLM Scoring Engine (Feb 16 – Feb 27, 2026)

Sprint 2 introduced the core AI scoring capabilities, integrating Groq LLaMA 3.1 8B for quality scoring, emotion detection, satisfaction percentage, and fairness analysis.

Task	Description	Team Member	Est. Hours	Status
T-09	Integrate Groq LLaMA 3.1 8B for quality analysis	Siddhi Dhamal	8	Completed
T-10	Build empathy, compliance, resolution scoring (1–10)	Siddhi Dhamal	8	Completed
T-11	Implement empathy timeline and compliance steps	Siddhi Dhamal	5	Completed
T-12	Build customer emotion detection (8 categories)	G. L. Gayathri	5	Completed
T-13	Implement satisfaction percentage scoring (0–100%)	G. L. Gayathri	5	Completed
T-14	Add bias/fairness analysis (4 sub-scores)	Sowmya	5	Completed
T-15	Save per-file scores to file_scores folder	Sowmya	3	Completed

6.3 Sprint 3 — RAG Pipeline & Dashboards (Mar 2 – Mar 13, 2026)

Sprint 3 was the most feature-rich sprint, delivering the full React dashboard, RAG pipeline with Pinecone, charts, batch upload, mobile responsiveness, and report generation.

Task	Description	Team Member	Est. Hours	Status
T-16	Build React dashboard with Vite + TailwindCSS	G. L. Gayathri / Siddhi	8	Completed
T-17	Add Recharts visualisations for all quality metrics	Sowmya	5	Completed
T-18	Implement call history with timestamp and summary	Sowmya	3	Completed
T-19	Build PDF and DOC report download feature	Siddhi Dhamal	5	Completed
T-20	Implement batch audio upload with async processing	Siddhi Dhamal	8	Completed
T-21	Make UI mobile responsive with bottom navigation	G. L. Gayathri	3	Completed
T-22	Add localStorage persistence for navigation state	G. L. Gayathri	3	Completed
T-23	Integrate LangChain + Pinecone for contextual retrieval	Siddhi Dhamal	8	Completed
T-24	Build RAG-powered audit summary endpoint	Siddhi Dhamal	5	Completed

6.4 Sprint 4 — Deployment & Monitoring (Mar 16 – Mar 24, 2026)

Sprint 4 focused on consolidating all backend servers, cloud deployment on Render and Vercel, setting up uptime monitoring, and final production fixes.

Task	Description	Team Member	Est. Hours	Status
T-25	Combine all FastAPI servers into single main.py	Siddhi Dhamal	5	Completed
T-26	Deploy backend on Render cloud platform	Siddhi Dhamal	3	Completed
T-27	Deploy frontend on Vercel	G. L. Gayathri	3	Completed
T-28	Set up UptimeRobot monitoring (5 min ping)	Sowmya	1	In Progress
T-29	Fix IST timezone for accurate Indian timestamps	Siddhi Dhamal	2	Completed
T-30	Add MIT Licence to GitHub repository	Sowmya	1	Completed
T-31	Fix per-file transcript for correct PDF/DOC reports	G. L. Gayathri	3	Completed

7. Stand-Up Meetings & Impediments

The team held daily stand-ups throughout all four sprints. Key impediments encountered and their resolutions are documented below.

Sprint	Day	Impediment	Action Taken
Sprint 1	Day 2	Lack of clarity on Deepgram API integration parameters	Conducted detailed discussion with mentors and team; responsibilities split clearly
Sprint 1	Day 4	Confusion on FastAPI routing between audio and text servers	Reviewed FastAPI documentation; split server responsibilities clearly
Sprint 2	Day 6	Groq LLM prompt formatting issues causing incorrect scores	Conducted team debugging session; standardised prompt templates
Sprint 2	Day 8	Difficulty identifying edge cases for emotion detection	Conducted research and tested with diverse audio samples
Sprint 2	Day 10	Fairness scoring sub-categories not aligned with requirements	Created priority matrix and reviewed with mentor
Sprint 3	Day 14	Recharts visualisations not rendering on mobile screens	Collaborated across team to debug responsive layout issues
Sprint 3	Day 16	Batch upload async processing causing timeout errors	Optimised async queue processing strategy
Sprint 3	Day 18	Pinecone index setup taking longer than expected due to embedding configuration	Collaborated to resolve Pinecone API config; RAG integration completed
Sprint 4	Day 22	Render cold-start delays affecting UptimeRobot ping results	Scheduled server warm-up strategy; adjusted ping interval

8. Sprint Retrospectives

After each sprint, the team conducted retrospective sessions to reflect on processes and improve collaboration. Key insights per sprint are summarised below.

8.1 Sprint 1 Retrospective

Team Member	Start Doing	Stop Doing	Continue Doing
Gunda Lakshmi Gayathri	Reviewing audio transcription output regularly	Assuming Deepgram API responses are always structured	Daily stand-ups and code reviews
Siddhi Dhamal	Documenting speaker diarisation edge cases	Delaying testing of boundary conditions	Maintaining clear API contracts
Sowmya	Prioritising text chat parser coverage	Rushing through summary integration	Asking for help when stuck on Deepgram config

8.2 Sprint 2 Retrospective

Team Member	Start Doing	Stop Doing	Continue Doing
Gunda Lakshmi Gayathri	Participating in LLM prompt engineering sessions	Delaying empathy score calibration	Collaborating on scoring rubric design
Siddhi Dhamal	Writing unit tests for emotion detection pipeline	Skipping edge-case testing for emotion categories	Maintaining focus on LLM output consistency

8.3 Sprint 3 Retrospective

Team Member	Start Doing	Stop Doing	Continue Doing
Full Team	Participating in dashboard design reviews	Delaying mobile responsiveness fixes	Collaborating on chart component reusability
Siddhi Dhamal	Integrating RAG pipeline early in sprint	Underestimating Pinecone setup complexity	Collaborating on LangChain prompt engineering

8.4 Sprint 4 Retrospective

Team Member	Start Doing	Stop Doing	Continue Doing
Full Team	Monitoring Render deployment logs proactively	Ignoring cold-start latency issues	Reviewing deployment checklists before releases

9. Individual Contribution — Gunda Lakshmi Gayathri

This section details the specific technical contributions made by Gunda Lakshmi Gayathri across all four sprints of the AuraQ project.

9.1 Sprint 1 Contributions

- Designed and built the FastAPI text server (chat_app.py) to handle text/chat log uploads end-to-end.
- Implemented a dynamic chat parser that normalises any speaker format (e.g., 'Agent:', 'Customer:', 'Support:', free-form) into a consistent structure before analysis.
- Integrated Deepgram Text Intelligence (summarise v2) to generate auto-summaries from normalised chat transcripts.
- Ensured per-file transcript and summary persistence to disk with correct file naming conventions.

9.2 Sprint 2 Contributions

- Built the customer emotion detection module — classifying customer sentiment across 8 emotion categories using LLM-based analysis.
- Implemented satisfaction percentage scoring (0–100%) with descriptive labels (e.g., Highly Satisfied, Neutral, At Risk) derived from the emotion and conversation analysis.
- Contributed to LLM prompt engineering sessions and calibration of empathy and emotion scoring parameters.

9.3 Sprint 3 Contributions

- Co-built the React + Vite + TailwindCSS dashboard (with Siddhi Dhamal), structuring the component architecture and integration with the FastAPI backend.
- Designed and implemented the mobile-responsive layout with a bottom navigation bar, ensuring seamless usability on smartphones and tablets.
- Added localStorage-based persistence for navigation state, so the app retains the user's current view across page reloads.
- Built the PDF and DOC report download feature, generating per-file audit reports with full transcript, summary, and quality scores.
- Fixed the per-file transcript linkage to ensure reports correctly reference the right call file's data.

9.4 Sprint 4 Contributions

- Led the consolidation of all FastAPI servers (audio, text, RAG, scoring) into a single unified main.py with merged route prefixes for a simplified deployment footprint.
- Deployed and configured the React frontend on Vercel, including environment variable setup and CORS alignment with the Render backend.
- Applied IST (UTC+5:30) timezone correction across all timestamp fields to ensure accurate Indian time display for call history records.

9.5 Contribution Summary

Sprint	Key Deliverable	User Stories	Tasks
Sprint 1	FastAPI text server, dynamic chat parser, Deepgram summarisation	US-003, US-004	T-04, T-05, T-06
Sprint 2	Emotion detection (8 categories), satisfaction scoring (0–100%)	US-008, US-009	T-12, T-13
Sprint 3	React dashboard, mobile UI, localStorage, PDF/DOC reports, report fix	US-011, US-014, US-016	T-16, T-19, T-21, T-22, T-31
Sprint 4	Server consolidation (main.py), Vercel deployment, IST timestamps	US-020, US-021, US-023	T-27, T-29, T-31

10. Deployment Architecture

10.1 Production Environment

Component	Platform	Details
Frontend (React SPA)	Vercel	Auto-deployed from GitHub; CDN-distributed globally. URL: aura-q-ai-customer-quality-auditor-eight.vercel.app
Backend (FastAPI)	Render	Single unified main.py service. Free-tier with cold-start management.
Uptime Monitoring	UptimeRobot	5-minute ping interval to keep Render instance warm. Status: In Progress
Vector Database	Pinecone Cloud	Managed vector store for RAG embeddings. Connected via LangChain.
AI Models	Groq Cloud + Deepgram	Groq for LLM inference; Deepgram for audio transcription.

10.2 Deployment Flow

- Developer pushes code to GitHub repository.
- Vercel auto-detects changes to the frontend (React) and triggers a build and deploy pipeline.
- Render deploys the FastAPI backend from the main branch; environment variables (API keys for Deepgram, Groq, Pinecone) are stored securely in Render's dashboard.
- UptimeRobot pings the Render health endpoint every 5 minutes to prevent cold-starts.
- The React frontend communicates with the Render backend via HTTPS API calls; CORS is configured to allow the Vercel domain.

11. Risks & Mitigations

Risk	Impact	Mitigation
Render cold-start delays (~50s) degrading user experience	High	UptimeRobot 5-min ping to keep instance warm; user-facing loading indicators added
Deepgram API quota exceeded during batch uploads	Medium	Async queue with rate limiting; per-file error handling to prevent full batch failure
Pinecone index misconfiguration breaking RAG pipeline	High	Tested configuration thoroughly in Sprint 3; embedding dimensions validated before production push
LLM hallucinations producing inaccurate quality scores	Medium	Prompt engineering standardisation; scores calibrated against sample calls during Sprint 2
Large audio files causing timeout on free-tier Render	Medium	File size limits enforced on frontend; chunked processing for large inputs
CORS errors breaking frontend-backend communication	Low	CORS middleware configured in FastAPI with explicit Vercel domain allowlist

12. Conclusion & Future Enhancements

12.1 Project Summary

AuraQ successfully delivered a fully functional, cloud-deployed AI Customer Quality Auditor in 8 weeks across 4 Agile sprints. The platform covers the complete QA pipeline — from raw audio/text input through AI-powered transcription, scoring, emotion detection, RAG-based retrieval, and report generation — all accessible through a modern, mobile-responsive React dashboard.

All 22 of 23 user stories were completed (US-022 UptimeRobot monitoring is in progress). The project demonstrates a strong integration of modern AI APIs (Deepgram, Groq, LangChain, Pinecone) within a clean FastAPI + React full-stack architecture, deployed on industry-standard cloud platforms.

12.2 Key Achievements

- End-to-end automation of the QA review process — from file upload to scored, downloadable report.
- RAG-powered intelligence enabling policy-aware scoring and contextual transcript Q&A.
- Bias and fairness module to ensure equitable, defensible agent evaluations.
- Production deployment on Vercel + Render with cloud-based uptime monitoring.
- Mobile-first responsive design for on-the-go access by supervisors.

12.3 Future Enhancements

- User Authentication — Role-based access control (admin, supervisor, agent) with secure login.
- Real-Time Call Monitoring — Live call quality scoring via websocket streaming.
- CRM Integrations — Connect with Salesforce, Zendesk, or Freshdesk to auto-import interactions.
- Advanced Analytics — Trend analysis, agent benchmarking, and predictive quality modelling.
- Multi-Language Support — Transcription and scoring in languages beyond English.
- PostgreSQL / Cloud DB — Migrate from CSV-based storage to a scalable database.
- Automated Alerts — Email/SMS notifications to supervisors when scores fall below thresholds.

—— End of Document ——